Linear Attention 101

Biao Zhang 张彪
Xi'an Jiaotong Univ & VAST
1zb.github.io

Vector Form

$$\frac{\sum_{i=1}^{M}\exp(q^\top k_i)v_i}{\sum_{i=1}^{M}\exp(q^\top k_i)} = \sum_{i=1}^{M} a_i \cdot v_i = \sum_{i=1}^{M}\frac{\exp(q^\top k_i)}{\sum_{j=1}^{M}\exp(q^\top k_j)}\cdot v_i \in \mathbb{R}^C$$

Matrix Form (Softmax Attention)

$$\mathrm{softmax}(QK^T)V$$

$\mathrm{softmax}\left(\underset{N\times C}{Q}\ \underset{C\times M}{K^\top}\right)\underset{M\times C}{V}$

$\underset{N\times M}{A}$ — Attn Matrix

Time Complexity: $O(N\cdot M\cdot C)$

Two-Layer MLP with Softmax as Act

$$f(Q) = \underbrace{\mathrm{act}(\overbrace{Q\cdot K^\top}^{\text{second layer}})}_{\text{first layer}}\cdot V \in \mathbb{R}^{N\times C}$$

Vector Form

$$\left(\sum_{i=1}^{M} v_i k_i^\top\right) q \in \mathbb{R}^C$$

Matrix Form (Linear Attention)

$$Q(K^\top V)$$

$\underset{N\times C}{Q}\left(\underset{C\times M}{K^\top}\ \underset{M\times C}{V}\right)$

Time Complexity: $O((N+M)\cdot C^2)$

Linear Layer

$$f(Q) = Q\cdot\underbrace{(K^\top V)}_{\text{Weight}} \in \mathbb{R}^{N\times C}$$

Non-Linear MLPs >> Single Linear Layers

Causal Self Attention $(N=M)$

Softmax Attention

- Train: $O = \mathrm{softmax}(QK^T\odot M)V$ — Parallel Form, Time: $O(N^2\cdot C)$
- Test:
$$(K_t, V_t) = \mathrm{Append}\left((K_{t-1}, V_{t-1}), k_t, v_t\right),$$
KV Cache
$$o_t = \mathrm{Read}(q_t, K_t, V_t), \quad = \mathrm{softmax}\left(q_t K_t^\top\right) V_t.$$
Recurrent Form, Cache: $O(t\cdot C)$, Cache Size is Increasing, Time t-th token: $O(t\cdot C)$, Time all tokens: $O(N^2\cdot C)$

Linear Attention → Train & Test
$$S_t = S_{t-1} + v_t k_t^T \quad \text{State ("KV Cache")}$$
$$o_t = \left(\sum_{i=1}^{t} v_i k_i^T\right)\cdot q_t = S_t \cdot q_t$$
Recurrent Form, State: $O(C^2)$, State Size is Fixed, Time t-th token: $O(C^2)$, Time all tokens: $O(N\cdot C^2)$

In-Context Learning (ICL)

KV Cache, State } updated during testing

LLMs

$$o_t = \mathrm{Layer}(q_t, K_t, V_t)$$

Output Token, Query Token

	RNNs	Linear Attns
input	x_t	q_t, k_t, v_t
state	$h_t = f(h_{t-1}, x_t)$	$S_t = S_{t-1} + v_t k_t^\top$
output	$y_t = g(h_t, x_t)$	$o_t = S_t \cdot q_t$

[1] Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention

Linear Attention

Training data: / Testing data:	$\{(k_1, v_1), (k_2, v_2), \cdots, (k_t, v_t), \cdots, (k_M, v_M)\}$ (Input, Label) / $\{q_1, q_2, \cdots, q_t, \cdots, q_M\}$
Function:	$f(k; S) = S\cdot k$
Loss (one sample):	$\mathcal{L}(f(k_i; S), v_i) = -v_i^\top\cdot(S\cdot k_i)$
Gradient:	$\nabla\mathcal{L}(S) = -v_t k_t^\top$
Training with SGD:	$S_t = S_{t-1} + \eta\cdot v_t k_t^\top$ with learning rate $\eta = 1$
Testing:	$o_t = f(q_t; S_t) = S_t\cdot q_t$

DeltaNet

Training data: / Testing data:	$\{(k_1, v_1), (k_2, v_2), \cdots, (k_t, v_t), \cdots, (k_M, v_M)\}$ / $\{q_1, q_2, \cdots, q_t, \cdots, q_M\}$
Function:	$f(k; S) = S\cdot k$
Loss (one sample):	$\mathcal{L}(f(k_i; S), v_i) = \frac{1}{2}\left\|S\cdot k_t - v_t\right\|_2^2$
Gradient:	$\nabla\mathcal{L}(S) = (S\cdot k_t - v_t)\cdot k_t^\top$
Training with SGD:	$S_t = S_{t-1} - \eta_t\cdot(S_{t-1}\cdot k_t - v_t)\cdot k_t^\top$
Testing:	$o_t = f(q_t; S_t) = S_t\cdot q_t$

[1] Linear Transformers Are Secretly Fast Weight Programmers
[2] Parallelizing Linear Transformers with the Delta Rule over Sequence Length

TTT (Test Time Training)

Training data: / Testing data:	$\{(k_1, v_1), (k_2, v_2), \cdots, (k_t, v_t), \cdots, (k_M, v_M)\}$ / $\{q_1, q_2, \cdots, q_t, \cdots, q_M\}$
Function:	$f(k; S) = \mathrm{MLP}(k; S)$
Loss (one sample):	$\mathcal{L}(f(k_i; S), v_t) = \frac{1}{2}\left\|\mathrm{MLP}(k_i; S) - v_t\right\|_2^2$
Gradient:	Autograd
Training with SGD:	
Testing:	$o_t = f(q_t; S_t) = \mathrm{MLP}(q_t; S_t)$

[1] Learning to (Learn at Test Time): RNNs with Expressive Hidden States

TTR (Test Time Regression)

Training data: / Testing data:	$\{(k_1, v_1), (k_2, v_2), \cdots, (k_t, v_t), \cdots, (k_M, v_M)\}$ / $\{q_1, q_2, \cdots, q_t, \cdots, q_N\}$
Training (Memory associations)	$f_t = \arg\min_{f\in\mathcal{M}}\frac{1}{2}\sum_{i=1}^{t}\gamma_i\left\|f(k_i) - v_i\right\|_2^2$
Testing (Memory retrieval)	$o_t = f_t(q_t)$

[1] Test-time regression: a unifying framework for designing sequence models with associative memory

From Perseptive of Bi-level Optimization

Inner Optimization: State: $\mathcal{S} = \{S\}$ — Updated during testing

Outer Optimization: Network: θ — Updated during training

Outer Loss
$$\underset{\theta}{\text{minimize}}\quad \mathcal{L}_{\text{outer}}(\theta, \mathcal{S}^*)$$
$$\text{subject to}\quad \mathcal{S}^* \in \underset{\mathcal{S}}{\arg\min}\ \mathcal{L}_{\text{inner}}(\theta, \mathcal{S}).$$
Inner Loss

Neural Architecteure Search

Architecteure

Outer Loss
$$\underset{\alpha}{\text{minimize}}\quad \mathcal{L}_{\text{val}}(w^*, \alpha)$$
$$\text{subject to}\quad w^* \in \underset{w}{\arg\min}\ \mathcal{L}_{\text{train}}(w, \alpha).$$
Inner Loss

Network Weights

[1] DARTS: Differentiable Architecture Search

Few Shot Learning

Slow Weights

Outer Loss
$$\underset{\theta}{\text{minimize}}\quad \mathbb{E}_{\mathcal{T}_i\sim p(\mathcal{T})}\left[\mathcal{L}_{\mathcal{T}_i}^{\text{qry}}\left(\theta, \phi_i^*\right)\right]$$
$$\text{subject to}\quad \phi_i^* \in \underset{\phi_i}{\arg\min}\ \mathcal{L}_{\mathcal{T}_i}^{\text{sup}}(\theta, \phi_i),$$
Inner Loss

Fast Weights

[1] Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks

TTT-E2E

$$\underset{\theta}{\text{minimize}}\quad \mathbb{E}_{\mathcal{P}\sim\mathcal{D}}\left[\mathcal{L}_{\text{outer}}\left(\theta, \phi^*(\theta, \mathcal{P}), \mathcal{P}\right)\right]$$
$$\text{subject to}\quad \phi^*(\theta, \mathcal{P}) \in \underset{\phi}{\arg\min}\ \mathcal{L}_{\text{inner}}(\theta, \phi, \mathcal{P}).$$

Plug-in MLPs; Cross Entropy, Not Regression Anymore!; Prompt/Context/Data

[1] End-to-End Test-Time Training for Long Context

In-Place TTT & TTT-NTP

$$\underset{\theta}{\text{minimize}}\quad \mathbb{E}_{\mathcal{P}\sim\mathcal{D}}\left[\mathcal{L}_{\text{outer}}\left(\theta, \phi^*(\theta, \mathcal{P}), \mathcal{P}\right)\right]$$
$$\text{subject to}\quad \phi^*(\theta, \mathcal{P}) \in \underset{\phi}{\arg\min}\ \mathcal{L}_{\text{inner}}(\theta, \phi, \mathcal{P}).$$

(Part) Network Weights; Loss on Latent Set; Prompt/Context/Data

[1] In-Place Test-Time Training
[2] Test-Time Training with Next-Token Prediction

1

Inner Loss on Train/Support Set

DARTS: Train Set; MAML: Support Set — $\{(x, y)\}$ Real

Inner Loss on Latent Set

Linear Attn: Latent Set — $\{(k, v)\}$ Synthesized

2

Single-Step GD

DARTS: $w \leftarrow w - \eta\nabla_w\mathcal{L}_{\text{train}}(w, \alpha)$

MAML: $\phi_i \leftarrow \theta - \eta\nabla_\theta\mathcal{L}_{\mathcal{T}_i}^{\text{sup}}(\theta)$

Multiple-Steps GD

Linear Attn (DeltaNet): $S_t = S_{t-1} - \eta_t\cdot(S_{t-1}\cdot k_t - v_t)\cdot k_t^\top$, $t = 1, 2, \cdots, M$

Continual Learning --> Recursive Self-Improvement

?

Implicit Inner Optimization: Softmax Attention —(Acceleration)→ Linear Attention —(Inner GD)→ DeltaNet —(Non-Linear Function)→ Test-Time Training(TTT)

Explicit Inner Optimization: Test-Time Updateing KVCache / States; Meta Learning; TTT-E2E / In-Place TTT / TTT-NTP; Test-Time Updating Network Weights

	Implicit	Explicit
Inner Optimization	Softmax Attention	DeltaNet, Linear Attention, Test-Time Training(TTT), TTT-E2E / In-Place TTT / TTT-NTP
Test-Time Updating	Softmax Attention, Linear Attention, DeltaNet, Test-Time Training(TTT)	TTT-E2E / In-Place TTT / TTT-NTP
	States	Network Weights